



Saint Columban College
College of Computing Studies
Pagadian City



vault-door-training

vault-door-training 



Easy Reverse Engineering picoCTF 2019

AUTHOR: MARK E. HAASE

Hints 

Description

(None)

Your mission is to enter Dr. Evil's laboratory and retrieve the blueprints for his Doomsday Project. The laboratory is protected by a series of locked vault doors. Each door is controlled by a computer and requires a password to open. Unfortunately, our undercover agents have not been able to obtain the secret passwords for the vault doors, but one of our junior agents obtained the source code for each vault's computer! You will need to read the source code for each level to figure out what the password is for that vault door. As a warmup, we have created a replica vault in our training facility. The source code for the training vault is here: [VaultDoorTraining.java](#)

90,148 users solved

 72% Liked 

 picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: vault-door-training

Category: Reverse Engineering

Date: 04/03/26



I. Objective

To analyze the given Java program and determine the correct password by inspecting its source code, then reconstruct it into the proper flag format for submission.

II. Background

In programming, a **hardcoded value** is something directly written inside the code.

Example:

- password inside the program
- API keys inside code

This is insecure because:

Anyone who reads the code can see the secret

In reverse engineering, we analyze programs to:

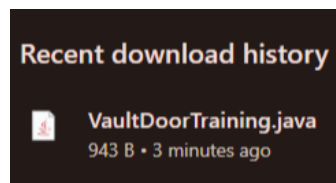
- understand logic
- find hidden data
- extract sensitive information

III. Tool Used

- **VS Code** - Used to read and analyze the Java program

IV. Methodology

1. Downloaded the provided Java source file





2. Opened the file using VS Code

```
1  import java.util.*;
2
3  class VaultDoorTraining {
4      public static void main(String args[]) {
5          VaultDoorTraining vaultDoor = new VaultDoorTraining();
6          Scanner scanner = new Scanner(System.in);
7          System.out.print(s: "Enter vault password: ");
8          String userInput = scanner.next();
9          String input = userInput.substring("picoCTF{".length(),userInput.length()-1);
10         if (vaultDoor.checkPassword(input)) {
11             System.out.println(x: "Access granted.");
12         } else {
13             System.out.println(x: "Access denied!");
14         }
15     }
16
17     // The password is below. Is it safe to put the password in the source code?
18     // What if somebody stole our source code? Then they would know what our
19     // password is. Hmm... I will think of some ways to improve the security
20     // on the other doors.
21     //
22     // -Minion ☐ #9567
23     public boolean checkPassword(String password) {
24         return password.equals(anObject: "w4rm1ng_Up_w1tH_jAv4_000iPnsaWOY");
25     }
26 }
27
```

3. Analyzed the program logic

4. Found this critical line:

```
public boolean checkPassword(String password) {
    return password.equals(anObject: "w4rm1ng_Up_w1tH_jAv4_000iPnsaWOY");
}
```

5. Identified this string as the correct password

6. Wrapped it in the required format:

picoCTF{w4rm1ng_Up_w1tH_jAv4_000iPnsaWOY}

7. Submitted the flag

V. Result

picoCTF{w4rm1ng_Up_w1tH_jAv4_000iPnsaWOY}



VI. Explanation

What is this program doing?

The program is like a **vault door**

It asks:

Enter vault password:

What happens to your input?

String input =

userInput.substring("picoCTF{".length(),userInput.length()-1);

This means:

- It removes:
 - **picoCTF{** at the start
 - **}** at the end

Example:

If you type:

picoCTF{hello}

It becomes:

hello

The Important Part

return password.equals("w4rm1ng_Up_w1tH_jAv4_000iPnsaWOY");

This means:

“The password must EXACTLY match this text”

So where is the secret?

It's literally written in the code

Think of it like:

A teacher says:

“The answer is hidden in this paper”



...and the answer is just written there LMAO

Why is this bad?

Because:

- Anyone can read the code
- Anyone can find the password
- No real security exists

Real-Life Analogy

Imagine:

- A safe
- The password is written on the door

Anyone can open it

Nevermind, maybe I'm bad at giving analogies hehays

VII. Conclusion

This challenge demonstrates how insecure it is to store sensitive data directly in source code.

By simply reading the program, the correct password was revealed without needing to guess or brute-force anything.

This reinforces an important cybersecurity principle:

Never hardcode secrets in your code.

— **The Analyst: Hyposelenia**